Interface-Scoped Dispatch: General Delegation with $O(1)$ Self-Calls

R. SCOTT MCKINNEY, Manifold Systems, USA

The conceptual appeal of object composition lies in its ability to build loosely coupled systems from independent runtime components. Yet conventional object composition lacks late-bound self-calls, or *open recursion*, a defining strength of implementation inheritance. Restoring open recursion while retaining the properties of object composition remains an unmet challenge.

We present *Parts*, a general delegation model that unifies independent runtime components, arbitrary composition, and open recursion. Its enabling mechanism, interface-scoped dispatch (ISD), assigns composite identity per interface at construction, allowing self-calls to dynamically dispatch through overriding composites. Parts is realized as a compiler plugin for Java 8–27 with IDE integration. It achieves $O(1)$ self-dispatch cost that is empirically equivalent to a standard vtable call, providing open recursion through composition without sacrificing the independence and flexibility that make object composition attractive.

1 Introduction

Object composition is the standard approach to structuring behavior from reusable components [1]. Most object-oriented languages support composition through some form of *forwarding* [16], providing an alternative to implementation inheritance and its well-known hazards, including fragile base classes and coupling to superclass internals. Forwarding, however, gives up a property that makes inheritance particularly useful: *open recursion*. With open recursion, a component's self-calls remain late-bound, allowing them to reach overrides supplied by a surrounding composition. Without it, self-calls are fixed to the component's local implementation and cannot reach such overrides. Thus, conventional composition preserves independent components but loses the dynamic self-dispatch that makes inheritance effective for extensible behavior.

The challenge is to recover open recursion without giving up the essential properties of object composition. A model that fully meets this challenge must provide three properties: *independent components*, meaning late-bound objects supplied at construction; *arbitrary composition*, meaning unrestricted composition topologies (Section 3.3); and *open recursion*, meaning self-calls that remain late-bound through the composite. The surveyed composition approaches achieve at most two of these three properties (Section 7).

The composition mechanisms of conventional languages differ in syntax and implementation but reduce to forwarding. Some, such as Rust and Swift, require forwarding to be written explicitly; others, including Kotlin's `by` clause [29] and Go's struct embedding [31], provide it automatically. These mechanisms support independent components under arbitrary composition, but forwarding alone cannot preserve the composite identity required for open recursion: a component's `this` remains the component itself, so its self-calls cannot be redirected through a composite. This is the *Object Schizophrenia* problem [4, 16].

Trait composition, including Scala's `with` [11] and mixins [5], makes the opposite trade. Self-calls through `this` dispatch within the composed hierarchy, so overrides remain visible. But traits and mixins are fused into the hierarchy at compile time and execute as a single runtime object. The constituent components therefore lose independent runtime identity, while composition is statically fixed rather than dynamically defined.

Several research projects (Section 7) have explored *delegation* [3, 15] as a means of retaining independent components while restoring open recursion. These models improve on conventional forwarding, but so far support only restricted composition topologies rather than the arbitrary

Author's Contact Information: R. Scott McKinney, Manifold Systems, USA, scott@manifold.systems.

topologies available through ordinary forwarding (Section 3.3). Many also introduce auxiliary mechanisms to maintain composite identity, which increase implementation complexity and, in some cases, incur substantial runtime overhead.

We present *Parts*, a general delegation model that resolves these long-standing limitations by combining independent runtime components, arbitrary composition, and open recursion. Its enabling mechanism, *interface-scoped dispatch* (ISD), uses *links* to assign composite identity per interface at construction to *parts*, components that participate in delegation. A part's self-calls then dispatch dynamically through the composites that claim the corresponding interfaces. Because identity is assigned per interface rather than per component, different interfaces of the same component may participate in different composites, allowing open recursion under arbitrary composition topologies while preserving independent runtime objects. Links also support non-part components, providing ordinary forwarding when delegation is not required, so delegation and forwarding can coexist within the same composite.

The model's guarantees are established constructively: the dispatch invariant and $O(1)$ self-dispatch cost follow by construction, and the safety rules by static enforcement in the compiler and IDE (Section 4). A mechanized metatheory is a natural complement left to future work (Section 6.5).

The core contributions are:

- **Parts**: a general delegation model in which independent runtime components can be composed arbitrarily, with partial delegation, open-recursive self-dispatch, and statically enforced interface contracts. The model is defined over standard interface hierarchies with a central dispatch invariant (Invariant 1) and composite-scoped encapsulation (`@internal`).
- **Interface-scoped dispatch (ISD)**: the enabling mechanism that assigns composite identity per interface at construction, allowing open recursion under arbitrary composition through partial delegation, where a composite may claim a proper subset of a part's interfaces, with self-dispatch resolved in $O(1)$ time.
- **An implementation**: an open-source compiler plugin [33] for Java 8–27 that realizes Parts with $O(1)$ self-dispatch at vtable speed, together with an IntelliJ IDEA plugin.

2 Overview

Consider a minimal example that shows `@part` and `@link` working in a concrete single-rooted composite.

```
interface Actor {
  void takeAction();
  void attack();
}

@part class Hero implements Actor {
  public void takeAction() { attack(); }
  public void attack() { println("Swing club!"); }
}

class Wizard implements Actor {
  @link Actor actor;
  Wizard(Actor actor) { this.actor = actor; }
  public void attack() { println("Cast spell!"); }
}

new Wizard(new Hero()).takeAction(); // "Cast spell!"
```

When `Wizard` is constructed, the compiler wires `Hero`'s composite identity for `Actor` to `Wizard`. A call to `takeAction()` dispatches to `Hero.takeAction()`, and the self-call `attack()` within it dispatches back through `Wizard`, invoking `Wizard.attack()`, not `Hero.attack()`.

This is the decorator pattern with one addition: the wrapped object's self-call routes back through the composite. A plain decorator would forward `takeAction()` to the `Hero`, where the `attack()` call would stay on `Hero` ("Swing club!"), losing the override.

Because the link field accepts any `Actor` at construction time, the part is a real late-bound object, not fused into the class hierarchy. A different `Actor` implementation can be substituted without changing `Wizard`.

Interface contracts are enforced at compile time. The link field has type `Actor`, and the compiler verifies that `Wizard` implements the delegated interface. A raw part may not escape the composite dispatch path: passing or assigning `this` as the concrete part type is a compile error; interface types are required (Section 4.5).

This example is single-rooted where `Wizard` is the sole composite for `Hero`, and one composite identity suffices. The next section shows where a single composite identity breaks down, and how assigning identity per interface fixes it.

3 The Model

3.1 Motivating example

The following example shows a part with interfaces rooted at different composites. `Hero` implements both `Combatant` and `Actor`, and two wrappers decorate it. A `Wizard` changes how it fights (`Combatant`), while a `Demon` partially possesses it, seizing its name and will (`Actor`) but leaving its trained combat untouched. As in the overview, the wrapped hero's self-calls route back through the wrappers; now two wrappers claim different interfaces of the same part. So from a single `Hero` instance, `attack()` must dispatch to `Wizard` and `name()` to `Demon`. A single composite identity cannot serve both. Assigning a separate composite identity per interface resolves this. Figures 1 and 2 show the structure.

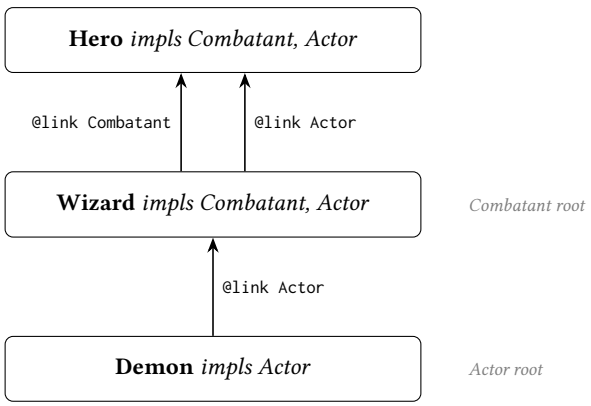


Fig. 1. Composition structure. Arrows show link fields labeled by the delegated interface. `Wizard` is the root for `Combatant` dispatch; `Demon` is the root for `Actor` dispatch.

```
interface Combatant {
  void attack();
}
interface Actor {
  String name();
  void takeAction();
}
@part class Hero implements Combatant, Actor {
  public void attack() { println("Swing club!"); }
  public String name() { return "Milo"; }
  public void takeAction() {
    print(name() + " acts: ");  // Actor dispatch
    attack();                   // Combatant dispatch
  }
}
@part class Wizard implements Combatant, Actor {
  @link Combatant combatant;
  @link Actor actor;
  Wizard(Hero h) { this.combatant = h; this.actor = h; }
  public void attack() { println("Cast spell!"); }
}
class Demon implements Actor {
  @link Actor actor;
  Demon(Actor a) { this.actor = a; }
  public String name() { return "Diablo"; }
}
Wizard wizard = new Wizard(new Hero()); // Combatant rooted at Wizard
Demon demon = new Demon(wizard);        // Actor rooted at Demon
demon.takeAction();                     // prints:  Diablo acts: Cast spell!
```

Fig. 2. Composition with multiple composite roots per part. `Hero` carries distinct composite identities for `Combatant` (rooted at `Wizard`) and `Actor` (rooted at `Demon`).

3.2 Definitions

Parts is defined over a standard typed object model where classes implement interfaces and method dispatch is defined by the receiver's runtime type.

claim an interface of a part is claimed when a composite links that interface, making the composite its exclusive identity within the part. A composite loses a claim when another composite later claims the same interface of that part.

component a class nominally implementing one or more interfaces, available for delegation if the class is a part and for forwarding otherwise.

composite a class containing one or more links. A composite nominally implements the interfaces declared on its links.

delegation surface a component's interface hierarchy, excluding any interface marked `@internal` in its `implements` clause.

interface an abstract type that defines a contract (a Java interface).

link an ordinary field annotated with `@link` and typed as an interface. The enclosing class (the composite) delegates the link's declared interface to the component assigned to the link at construction. Links are implicitly `private` and `final`; explicit modifiers are redundant.

part a component annotated with `@part`, enabling compiler plugin support for open recursion.

partial delegation a natural property of ordinary forwarding, allowing a composite to claim only some of a part's delegation surface and leaving the remaining interfaces available for other composites to claim. This is the basis of arbitrary composition (Section 3.3).

root (of an interface) the composite currently holding the claim on an interface of a part.

When a composite links an interface to a part, the part is directly wired to the composite. It records a *composite identity*: a reference to the composite statically indexed by the interface. The part's self-calls route through the composite indexed by the interface of the called method. The enabling mechanism is *interface-scoped dispatch* (ISD).

A part may itself contain link fields, in which case composite identity propagates transitively. When such a part is wired to an outer composite, that composite becomes the dispatch root for each interface it claims throughout the delegation chain, while interfaces it does not claim retain the roots established by inner composites. This *transitive propagation* continues through a link field only when the field's value is a part and the field's declared interface and the claimed interface are comparable, one a subtype of the other (Figure 6).

Composites must also resolve *diamonds*. When a composite has multiple links that share a common interface, a sharing link must be chosen via `@link(share=I.class)`, explicitly if ambiguous, at one of the link sites. This determines the leg of the diamond the linking composite forwards through for its implementation of the interface. Diamonds consisting of multiple shared interfaces require the same `share=I.class` to select the sharing links for each interface, where a single link can share multiple interfaces or the interfaces can be shared separately from any of the other links capable of sharing them. Wiring is unaffected: self-calls on common interfaces from other legs of the diamond naturally route through the shared leg.

Two practical extensions, composite-scoped encapsulation (`@internal`) and abstract parts with deferred linkage (`asLink()`), are defined and implemented (Section 4).

Dispatch rule. Traditional object-oriented dispatch resolves a call through the receiver's object identity:

$$\text{call } m() \text{ on } o \longrightarrow \mathit{vtable}(o,\, m)$$

Interface-scoped dispatch replaces the single object identity with a per-interface composite identity. A self-call $m()$ in part P where $m \in I$ resolves through the composite identity for I:

$$\text{self-call } m() \in I \text{ in } P \longrightarrow \$\mathit{selves}_P[I].m()$$

For self-calls within a part's methods, the dispatch triple is *(part, interface, method)* rather than the conventional *(object, method)*. Figure 3 contrasts this per-interface routing with a single shared composite identity. This is *open recursion*: a method's self-calls late-bind to the most-derived receiver. Here the receiver resolves from the interface of the self-call. Forwarding is the closed case: a component's self-calls stay in the component, and the composite never enters the recursion.

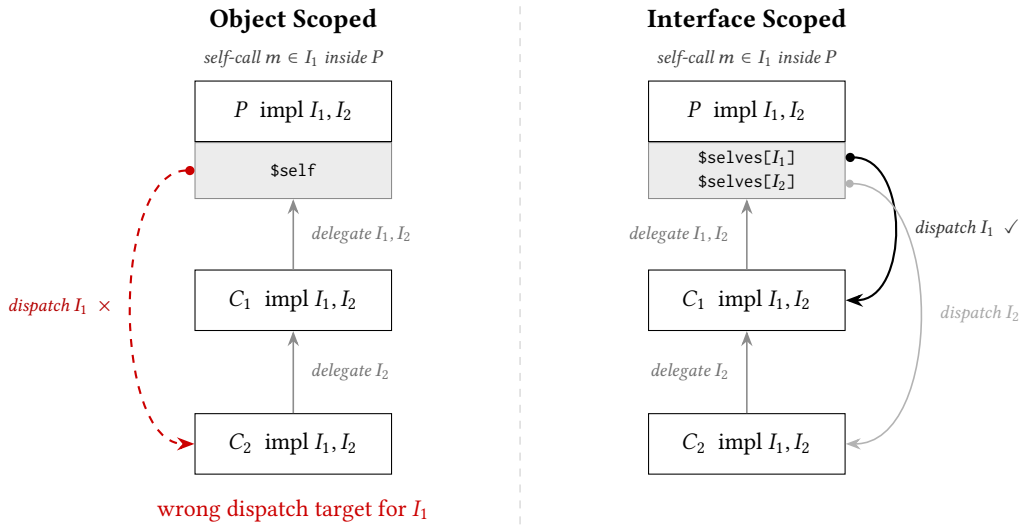


Fig. 3. Object-scoped vs. interface-scoped dispatch. In both panels C_2 (impl I_1, I_2) delegates only I_2 to C_1 (impl I_1, I_2), which delegates both interfaces to part P. *Left:* a self-call $m() \in I_1$ inside P routes through `$self` = C_2, the wrong composite for I_1. *Right:* `$selves[`I_1`]` routes to C_1; `$selves[`I_2`]` routes to C_2; each self-call reaches its correct composite root.

INVARIANT 1. *For any part P in a delegation chain for interface I rooted at composite C: a self-call* `this.m()` *within P where $m \in I$ dispatches to C.*

Composite identity is single-valued per interface. A part's `$selves` slot for I holds exactly one composite, established at construction, with the most recent wiring of I determining it. The root C in the invariant denotes this current occupant and is therefore always well-defined. Whether a given wiring order matches the programmer's intent is, as with any construction-time assignment, the programmer's responsibility.

The invariant is established by construction. When composite C establishes a link to part P for interface I, the plugin-generated `$linkPartToSelf` (Figure 6) sets $\$selves_P[I] \leftarrow C$ and recurses into P's own links, propagating C's identity for each interface C claims and leaving all other slots unchanged. Each interface of a part resolves through a single `$selves` slot. Wiring an outer composite that claims I re-roots this slot to that composite and leaves the slots for interfaces it does not claim unchanged, so the slot holds exactly one root at any time and stabilizes once the outermost composite claiming I is wired; C denotes that final root. Each write occurs during the wiring composite's construction. Wiring happens solely as link field assignment, and the plugin enforces `final` on link fields, so writes are confined to construction. The `$selves` slots are therefore never written after the outermost composite claiming I is constructed.

Visibility follows from safe publication. Once the composite root is safely published, the Java Memory Model guarantees that any thread observing it sees all writes that happened-before publication, the wired slots included. The role of `final` here is write-confinement, not the JMM's final-field freeze. Since array-element writes are not covered by freeze semantics, the guarantee rests on write-confinement together with safe publication of the root, not on the finality of the slots.

Self-calls in P where $m \in I$ are rewritten by the plugin to `((I) $selves[IDX_I]).m()`. For self-calls through the delegation surface, there is no code path in a part method that dispatches through

anything other than the indexed slot. Because the slot holds C once the composition is constructed and is not written thereafter, the invariant holds at every point a self-call can execute. Before wiring, each slot is initialized to `this`, so an unwired part dispatches on itself, consistent with standard object-oriented dispatch. In this initial state, the invariant holds trivially. A full mechanized proof in a Featherweight Java subset is a natural direction for follow-on work (Section 6.5).

3.3 Arbitrary Composition

Mirroring the structural flexibility of standard object composition and forwarding, Parts supports unrestricted topologies expressed as behaviorally acyclic graphs where nodes are parts, composites, or ordinary objects, and edges are link fields. Each edge establishes a connection from a composite to a component through the interface specified by the link.

Partial delegation permits a composite to claim only the interfaces it needs from a part's delegation surface, leaving the remaining interfaces for other composites to claim. This freedom gives rise to the four composition topology primitives shown in Figure 4 and classified over two dimensions: the number of roots a part maintains and the number of sources the composition has. A root is an outermost node claiming one or more interfaces of a part's delegation surface; a source is a root with no incoming link edges.

(a) **Single-root, single-source.** Composite A implements I_1, I_2 and delegates both to B, which implements both directly. A's links claim all of B's delegation surface, so B has one root, A, for both interfaces, and one source. This is the topology typical of decorator chains.

(b) **Multi-root, single-source.** Composite A implements I_1, I_2 and delegates only I_1 to B, which implements I_1, I_2 directly. Because A does not claim B's I_2, I_1 is rooted in A, while I_2 is rooted in B: one source, two roots. A and B in turn may delegate their free interfaces to separate components that participate in another subgraph that likewise decomposes into one of these four topologies. The motivating example discussed previously (Figure 1) is an instance of this topology combined with (a) where B delegates both its I_1 and I_2 to another part, propagating A and B as the part's roots for I_1 and I_2.

(c) **Multi-root, multi-source.** Component C implements I_1 and I_2, where A claims I_1 and B claims I_2. As in (b), C has different roots for its two interfaces, but here A and B are independent sources. This echoes subject-oriented programming's independent *subjects*, composed with no single subject designated as owner (Section 7). Figure 5 exercises this directly.

(d) **Cyclic, multi-root, sourceless.** Composite A implements I_1, I_2 and delegates I_1 to B, which implements I_1, I_2 and delegates I_2 back to A. Such structural cycles are permitted when delegation through them is behaviorally acyclic because each traversal proceeds through a distinct interface: I_1 is rooted in A, and I_2 is rooted in B. Cycles over nondistinct interfaces are rejected (Section 4).

Each of the four topologies illustrated takes its reduced form using as few nodes and edges as possible while retaining the essential properties that make the topology distinct. Topologies (a) and (b) each have a single source, but (b) is multi-root; (c) is multi-root and multi-source; (d) is multi-root and sourceless. Topology (d) is structurally cyclic but behaviorally acyclic over distinct interfaces.

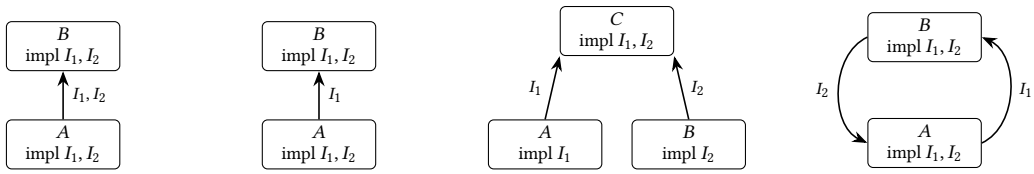
These four are exhaustive over the two dimensions. Of the six combinations the dimensions admit, two cannot occur: in reduced form, a single-rooted part forces a single source, and a sourceless graph is necessarily multi-root. This leaves (a)–(d).

Forwarding admits these topologies, so by definition a general delegation model must as well. The topologies therefore provide a basis for verifying whether a delegation model supports arbitrary

composition: does its treatment of open recursion remain correct under arbitrary extensions of each topology?

To answer this question, another set of labeled edges may be drawn onto the structural delegation graph to form a second graph in which each part is connected to the composites claiming its interfaces, determined by following the delegation edges backwards along each interface of the part's delegation surface to the outermost composite claiming the interface. This derived identity graph reveals what open recursion under arbitrary composition requires: composite identity assigned per interface.

In all four cases, for each delegated interface, a part acquires the identity of the composite claiming that interface, independent of where execution enters the composition. Invariant 1 holds independently for each interface.



(a) single-root, single-source
I_1, I_2 root: A

(b) multi-root, single-source
I_1 root: A; I_2 root: B

(c) multi-root, multi-source
I_1 root: A; I_2 root: B

(d) multi-root, sourceless,
I_1 root: A; I_2 root: B

Fig. 4. The four primary composition topologies under partial delegation. Arrows point from a delegating composite to the part it links. (a) and (b) have a single source; (c) has two sources; (d) is sourceless.

```
interface Actor { void takeAction(); int timeLimit(); }
interface Combatant { int strengthLevel(); }

@part class BasicHero implements Actor, Combatant {
  private int elapsedTime = 11;
  public void takeAction() { println("Attack with strength: " + strengthLevel()); }
  public int timeLimit() { return 10; }
  public int strengthLevel() { return outOfTime() ? 0 : 8; }
  private boolean outOfTime() { return elapsedTime > timeLimit(); }
}
class PaidSubscriber implements Actor {
  @link Actor actor;
  PaidSubscriber(Actor actor) { this.actor = actor; }
  public int timeLimit() { return 100; }
}
class SuperSoldierAddon implements Combatant {
  @link Combatant combatant;
  SuperSoldierAddon(Combatant combatant) { this.combatant = combatant; }
  public int strengthLevel() { return combatant.strengthLevel() * 10; }
}
BasicHero basicHero = new BasicHero();
PaidSubscriber subscriber = new PaidSubscriber(basicHero);
SuperSoldierAddon addon = new SuperSoldierAddon(basicHero);
subscriber.takeAction(); // reaches SuperSoldierAddon's strengthLevel()
// PaidSubscriber and SuperSoldierAddon hold no reference to each other.
// prints: Attack with strength: 80
```

Fig. 5. Topology (c): two independent composites, `PaidSubscriber` and `SuperSoldierAddon`, claim different interfaces of the same `BasicHero` with no common composite. The call `subscriber.takeAction()` reaches self-calls through the respective roots: `strengthLevel()` through `SuperSoldierAddon` and `timeLimit()` through `PaidSubscriber`.

4 Implementation

The implementation is a javac plugin that operates directly on the compiler's AST before and after two phases: *Enter* (symbol definition) and *Analyze* (semantic processing).

For the Enter phase, the plugin generates structural scaffolding. Here, `@part` classes are processed to add the `$selves` identity array, the `$linkPartToSelf` wiring method, and if the class is abstract, the `$Impl` subclass and `asLink()` factory methods. Composite classes are also processed here to add forwarding methods corresponding with `@link` delegation. At Analyze, `@part` classes are processed to rewrite self-calls and to enforce self-preservation rules (Section 4.5). `@link` field assignment is also rewritten here to invoke wiring, and to enforce `@internal`.

4.1 Part Transformation

Each part holds a `$selves` array with one slot for each interface on its delegation surface. The ordering of the array's slots is compile-time-determined according to the part's class information.

```
private final Object[] $selves = {this, ..., this};
// after wiring: $selves[IDX_I] = composite identity for interface I
```

Array slots are initialized with `this`, so a part's self-calls dispatch on itself by default.

Within a part, every call `this.m(args)` where `m` is declared in interface *I* is rewritten to:

```
((I) $selves[IDX_I]).m(args)
```

The cast ensures interface method invocation via the `invokeinterface` instruction, allowing the call to dispatch through the composite's implementation of that interface.

The ordinal `IDX_I` is assigned per part over its own delegation surface, so it is local to that class: the rewritten self-calls and the generated `$linkPartToSelf` both reside in the part and share its indexing, and no index agreement across separately-compiled classes is required.

```
public void $linkPartToSelf(Object root, Class<?>[] linkScope) {
  // cycle detection
  if (this == root) Internal.reportCycle(this, root);
  // wire the part's 'linkScope' interfaces to 'root'
  outer:
  for(Class<?> linkIface : linkScope) {
    for(int i = 0; i < $INTERFACE_CLOSURE.length; ++i) {
      Class<?> iface = $INTERFACE_CLOSURE[i];
      if (iface == linkIface) {
        this.$selves[i] = root;
        continue outer;
      }
    }
    throw new DelegationLinkageError("Unimplemented linked interface: " + linkIface);
  }
  // recursively call $linkPartToSelf to rewire the part's related links to 'root'
  Class[] scopeLinkField1 = Internal.widest($LINK_FIELD_1, linkScope);
  if (scopeLinkField1.length != 0 && this.linkField1 instanceof $PartClass) {
    (($PartClass)this.linkField1).$linkPartToSelf(root, scopeLinkField1);
  }
  // likewise for the linkField2, etc.
}
```

Fig. 6. Generated `$linkPartToSelf` for a part class. `Internal.widest` returns the wider of two narrowest-first ordered interface arrays, or an empty array if the arrays are not comparable.

4.2 Composite Transformation

A composite nominally implements every interface claimed by its links. For each method of a claimed interface that the composite does not implement directly, the plugin generates a delegating method that forwards through the corresponding link field:

```
R m(args) { return link.m(args); }
```

Each link field assignment `this.link = part` is transformed to invoke `part.$linkPartToSelf(this, I.class)`, wiring the part's `$selves` slot for the link's declared interface *I* to the composite. If the part itself links that interface to another part, the generated method recursively rewires that linked part to the same composite (Figure 6).

4.3 Abstract Part Support

A part may be abstract, deferring some interface methods to the composite. Linking an abstract part obligates the composite to implement those methods or to be declared abstract. When enforceable at compile time, the plugin generates a delegating method for each interface method implemented by the part, but none for methods it leaves abstract. Without those methods, a concrete composite is flagged by the compiler to either implement them directly or declare itself abstract. This is the compositional analog of the rule that a concrete class must implement its inherited abstract methods.

To make an abstract part instantiable for linking, the compiler generates a private `$Impl` inner subclass with stubs that throw `DelegationLinkageError`, and an `asLink()` factory per constructor:

```
private static final class $Impl extends AbstractClass {
  $Impl(/* AbstractClass ctor params */) { super(...); }
  @Override public R m(/* params */) {
    throw new DelegationLinkageError("Abstract method called");
  }
}
public static Foo asLink(/* ctor params */) { return new $Impl(...); }
```

Once wired, the stubs are unreachable for a statically configured abstract part. Self-calls dispatch through `$selves` to the composite, where the compile-time obligation guarantees an override. The stubs enforce the runtime configured case when the linking composite does not override any of the part's abstract methods. The stubs also serve as a runtime guard for direct abstract method calls on an `asLink()` instance.

4.4 Default Methods

If left unmitigated, a default method runs with the part as receiver, rather than the composite, when neither the composite nor the part overrides it, resulting in incorrect self-call dispatch.

A default method cannot be directly invoked with a receiver other than the current `this`. The implementation uses a method handle bound to an `invokedynamic` call site to work around this limitation. When a part does not override a default, the plugin generates an override that invokes the interface's default implementation in this way, with the receiver taken from the `$selves` slot of the interface that *provides* the default implementation, which may differ from the interface that declares the method. The default body then runs with a `this` that implements the default's interface, so its self-calls dispatch through the composite claiming that interface, as a self-call on that interface would.

```
interface Foo {
  void bar();
  default void foo(P p) { bar(); } // self-call
}
@part class FooPart implements Foo {
  public void bar() { ... }
  // generated
  // calls Foo.super.foo() with composite in $selves as receiver
  public void foo(P p) {
    invokedynamic Foo::foo((Foo) $selves[IDX_Foo], p);
  }
}
```

The site's bootstrap links it once to a fixed target (the default implementation), with the receiver as its only varying argument. Being monomorphic, it is inlined by the JIT to the same type of $O(1)$ array-indexed invocation as in Section 5.

4.5 Safety Guarantees

Cycle detection. If a part is wired to itself, directly or transitively, over the same interface, `DelegationLinkageError` is thrown at assignment time during the recursive `$linkPartToSelf()` call (Figure 6).

`@internal` *enforcement.* `@internal` may annotate an interface method or an interface type literal in a class's `implements` clause, with distinct effects. A method annotated with `@internal` is accessible only within the composite, the same intent behind `protected` in implementation inheritance, but more cleanly scoped for composition. A type literal in a class's `implements` clause annotated with `@internal` removes that interface from the class's delegation surface: no composite may claim it via a link field.

Self-preservation. Inside a part, the compiler restricts every use of `this`, implicit or explicit, that would expose the part's concrete identity outside the composite dispatch path. `this` may appear only (1) as a call or field-access receiver, where interface-method calls route through ISD to the composite while every field access and non-interface call runs directly on the part; (2) as an interface-typed value in any other position, rewritten to the current occupant of that interface's slot, either the claiming composite or, when the interface is unclaimed, the part itself; and (3) as the operand of `synchronized`, which retains no reference. An escaping interface-typed `this` thus takes the same path as an interface self-call, so a callee observes the same composition an external client would. Callbacks through the part's link fields, the compositional analog of `super`, are likewise permitted. Every other occurrence, whether direct, inferred, or captured, is rejected, including passing or assigning `this` as its concrete part type and casting it to `Object`.

A caller holding the bare part could invoke methods off the composite dispatch path and violate Invariant 1, so the escape must be prevented and no legitimate use should require it. A part has no outward identity beyond its interfaces, so a reference to the raw component, rather than to an interface it satisfies, adds nothing the composite does not already provide. Letting one escape could only split a single logical object in two, reintroducing the Object Schizophrenia that ISD otherwise eliminates.

Ambiguous self-calls. A bare self-call in a part is ambiguous when its method is carried by two or more mutually unrelated interfaces (neither assignable to the other), including when both inherit it from a common super-interface. The receiver must then be cast-qualified as the intended interface (Figure 7). Casting is otherwise optional. When a sub-interface and its super-interface are claimed separately, for instance, ISD resolves a bare self-call through the super-interface, though a part can still reach the sub-interface's override by casting to it. A cast-qualified `this` thus lets a part target separate overrides of the same method, which the multi-rooted topologies (b)–(d) (§3.3) require.

```
interface A { String a(); String z(); }
interface B { String b(); String z(); }
@part class AB implements A, B {
  public String a() { return ((B)this).z() + " " + ((A)this).z(); }
  public String b() { return ((A)this).z() + " " + ((B)this).z(); }
  public String z() { return "AB.z"; }
}
class AImpl implements A {
  @link A a;
  AImpl(A a) { this.a = a; }
  public String z() { return "AImpl.z"; }
}
class BImpl implements B {
  @link B b;
  BImpl(B b) { this.b = b; }
  public String z() { return "BImpl.z"; }
}
AB ab = new AB();
AImpl a = new AImpl( ab );
BImpl b = new BImpl( ab );
println(a.a()); // "BImpl.z AImpl.z"
println(b.b()); // "AImpl.z BImpl.z"
```

Fig. 7. Self-calls to overlapping method `z()` in interfaces `A` and `B` must be cast-qualified to the intended interface. Here, part `AB` reaches both implementations of `z()` by selecting the target interface with casting.

5 Evaluation

5.1 Self-Dispatch Performance

Self-dispatch through a composite compiles to a fixed sequence: an array-indexed load of `$selves[IDX_I]` (constant index), a cast to I, and an `invokeinterface`. This reaches the composite claiming the interface in $O(1)$, independent of composition depth, with the same shape as a vtable call. We measure with JMH [32] (average-time mode; 10 forks, 5 warmup and 10 measurement iterations per fork; ± is the 99.9% CI half-width), run on an Intel Core i7-1365U under Windows 11 with OpenJDK 21. Results are from a single machine, which we note as a limitation given the sub-nanosecond magnitudes; the claims below rest on measured bounds on the differences rather than exact figures.

Self-dispatch costs one interface call. Holding a target method non-inlinable so a real call occurs, we compare three depth-0 forms over the same receiver: `invokevirtual`, the `invokeinterface` it compiles to, and the plugin's emitted `((I) $selves[IDX_I]).m()` (an `Object[]` load, a cast, and `invokeinterface`). The plugin emits `invokeinterface`, so that is the comparison that isolates ISD: the two differ by 0.03 ± 0.09 ns, bounding the array load and cast to at most 0.12 ns at 99.9% confidence. Measured directly against `invokevirtual`, the emitted form differs by 0.10 ± 0.05 ns: the ordinary interface-versus-virtual dispatch gap, not a cost of ISD. Self-dispatch therefore costs a single interface call, within 0.15 ns of a vtable call at 99.9% confidence.

Self-dispatch does not drive depth scaling. To check how self-dispatch cost varies with composition depth, we sweep chain depth d (1–6). The *baseline* traverses d delegation hops to a leaf that returns directly; the *concrete* and *default* variants add one self-call through the composite root, issued

from a concrete part method and from an inherited interface default routed through the generated `invokedynamic` override of Section 4.4. Because both variants share the d hops, the delta (Table 1) is the added self-dispatch. It is bounded (0.55–0.84 ns) and grows by only about 0.06 ns per hop, small beside the roughly 0.35 ns per hop the traversal itself costs, so the self-call does not meaningfully add to depth scaling. Its receiver comes from a dynamically wired `$selves` slot, so it is not statically known and the dispatch does not fold away; the delta's size, and its slight upward drift, plausibly reflect the small body the self-call invokes and less complete inlining at greater depth. The *default* column tracks *concrete* to within 3% at every depth, so the `invokedynamic` default path adds no measurable cost over a concrete self-call; its one-time site bootstrap is absorbed during warmup.

Table 1. Self-dispatch cost by delegation depth (JMH, average-time mode; ns/op, ± 99.9% CI half-width). *Baseline* traverses the d delegation hops to a leaf that returns directly, with no self-call. *Concrete* and *Default* traverse the same hops and add one self-call through the composite root, issued from a concrete part method and from an inherited interface default (via `invokedynamic`), respectively. The delta (Concrete minus Baseline) is the cost of one self-dispatch over plain delegation: resolving the composite root through `$selves` and issuing the call to it. It is an upper bound on the dispatch increment, since it also includes executing the invoked method, which is a small fixed body at every depth and not a cost introduced by ISD. The delta is bounded and grows by about 0.06 ns per hop, far below traversal's roughly 0.35 ns per hop. Values are rounded from measured means and intervals; the delta is computed from the unrounded means (half-widths combined in quadrature) and may differ from the rounded column difference by 0.01 ns.

Depth	Baseline (ns)	Concrete (ns)	Default (ns)	Delta (ns)
1	0.62 ± 0.02	1.16 ± 0.03	1.18 ± 0.03	0.55 ± 0.03
2	0.91 ± 0.03	1.56 ± 0.04	1.55 ± 0.04	0.65 ± 0.05
3	1.18 ± 0.03	1.93 ± 0.05	1.92 ± 0.05	0.75 ± 0.06
4	1.64 ± 0.03	2.30 ± 0.06	2.23 ± 0.06	0.66 ± 0.07
5	1.93 ± 0.06	2.70 ± 0.06	2.76 ± 0.07	0.77 ± 0.09
6	2.37 ± 0.06	3.21 ± 0.09	3.23 ± 0.08	0.84 ± 0.11

We measure the delta against Parts' own forwarding path rather than a competing system: that path is semantically equivalent to Kotlin by and Scala 3's `export` (Section 7), so the delta prices exactly the self-dispatch routing those mechanisms cannot express. A cross-system benchmark would price implementation-strategy differences rather than this dispatch, so that comparison is analytic (Section 7).

Construction cost. Wiring is paid once, at construction, off the dispatch path. Each link assignment involving a part calls `$linkPartToSelf` (Figure 6), which sets the part's claimed slots and recurses through its link fields: work bounded by the part's interface surface, which is small in practice, and re-propagation on each wrap makes a chain of n composites $O(n^2)$ overall. Both are off the dispatch path and small for the shallow chains typical of composition, so the per-call cost is unchanged.

5.2 Example: Teaching Assistant

The following example exercises diamond dispatch resolution and self-call routing through the composite. A teaching assistant (TA) is both a student and a teacher where both roles share a common `Person` base.

```
interface Person {
  String name();
  String title();
  String titledName();
```

```
}
interface Student extends Person { ... }
interface Teacher extends Person { ... }
interface TA extends Student, Teacher { ... }

@part abstract class PersonPart implements Person {
  private final String name;
  public PersonPart(String name) { this.name = name; }
  public String name() { return name; }
  public String titledName() { return title() + " " + name(); }
}

@part class StudentPart implements Student {
  @link Person person;
  public StudentPart(Person p) { this.person = p; }
  public String title() { return "Student"; }
}

@part class TeacherPart implements Teacher {
  @link Person person;
  public TeacherPart(Person p) { this.person = p; }
  public String title() { return "Teacher"; }
}

@part class TaPart implements TA {
  @link(share=Person.class) Student student;
  @link Teacher teacher;
  public TaPart(Student student) {
    this.student = student;
    this.teacher = new TeacherPart(student);
  }
  public String title() { return "TA"; }
}

TA ta = new TaPart(new StudentPart(PersonPart.asLink("Milton")));
ta.titledName(); // "TA Milton"
```

With forwarding, the shared `Person` methods must each be resolved by hand, and `titledName()` still calls `title()` on the delegate's own `this`. Parts resolves both problems.

`PersonPart` holds the template once. Its self-call to `title()` dispatches through the composite at each level of the delegation chain.

`titledName()` is defined once in `PersonPart` and not overridden. Diamond resolution is handled statically: `@link(share=Person.class)` designates `student` as the forwarding link for `Person`, `TaPart` transitively claims `Person` throughout the composite.

6 Discussion

6.1 Partial delegation and part identity

Partial delegation is the basis of arbitrary composition, where a composite may claim only some of a part's delegation surface, as forwarding naturally allows. *Multi-rooted parts* result from this condition: a part's interfaces may be rooted at two or more distinct identities, including the part itself and one or more composites, via any combination of the multi-rooted topologies in Section 3.3.

The possession example in Section 3 demonstrates an extended version of topology (b): `Demon` claims only `Actor` from `Wizard`. Through delegation propagation, `Hero` consequently has separate roots for `Actor` and `Combatant`, making it a multi-rooted part through a single source graph, `Demon`. The PaidSubscriber/SuperSoldierAddon example (Figure 5) demonstrates topology (c): multiple roots within a multi-source composition.

This multi-rootedness makes a part's identity context-sensitive. References to `this`, whether implicit or explicit, resolve according to the interface through which they are used. When that interface is claimed, ISD resolves `this` to the claiming composite. Otherwise, `this` remains the part itself. This is a direct resolution of the Object Schizophrenia problem mentioned in Section 1.

6.2 Equality

Parts does not generate `equals` or `hashCode`. If object identity is insufficient for a composition's equality or hashing checks and state must be taken into account, we suggest the conventional formula for both part and non-part classes: require that the argument's class is exactly the receiver's class (`getClass()`), compare the remaining owned state, including link fields, and derive `hashCode` from the same state. This is the same strategy used with ordinary decorators.

6.3 Language Design Implications

Parts addresses several longstanding limitations of implementation inheritance:

- **Fragile base class** [2]: a composite's dependency surface is the interface contracts it explicitly claims, not a component's internal implementation; changes to its encapsulated internals do not propagate. Partial delegation completes the isolation: a composite is unaffected by changes to interfaces it does not claim.
- **Hierarchy-bound**: implementation inheritance binds behavior to a class hierarchy and a single object identity, making independently rooted compositions of the same component impossible. Parts permits arbitrary topologies by assigning composite identity per interface.
- **The *Object Schizophrenia* problem [4]**: a hazard with forwarding where self-calls remain within the component rather than routing through the composite; resolved by ISD.
- **Diamond ambiguity**: Parts provides explicit `@link(share=I.class)` resolution; ambiguous diamonds left unresolved are compile errors rather than being silently merged.
- **Unintended interface exposure**: a composite may implement a subset of its parts' interfaces; those it does not implement are absent from its API by construction. `@internal` strengthens this further: a class may declare selected interfaces as internal to its implementation and excluded from its delegation surface, preventing the interfaces from being claimed by a composite. At a more granular level, individual interface methods may be marked `@internal` to restrict their access to composite internals.
- **Abstract behavior**: abstract parts with deferred linkage (via `asLink()`) preserve the template-method capability of abstract classes without coupling their implementation and hierarchy.

Implementation inheritance's power is open recursion. Its fragility is the surface that power ranges over: *every accessible member*. Parts retains open recursion but bounds the surface to the declared interface contract. Self-calls still reach the composite's overrides, while the internals that make base classes fragile are never exposed.

Implementation inheritance and Parts are, essentially, two bindings of the same open recursion: implementation inheritance binds it to the class hierarchy, where a subclass IS-A its superclass and inherits its entire implementation surface, while Parts binds it to a held object and its delegation surface, where the composite HAS-A its parts and IS-A only its declared interfaces. The concept

that inheritance and open recursion are separable is Cook's [6, 7], and the modeling of inheritance as composition is Bracha and Cook's [5]. Cook's model threads self as an explicit parameter; Parts wires composite identity per-interface at construction.

6.4 Concurrency

A constructed composition is safe to share across threads. Each interface's `$selves` slot may be written during construction as links are established; the last write determines the resulting root. Once construction is complete, the slots are never written again, so the JMM makes the final wired state visible to any thread observing the safely published root. No synchronization is required on the dispatch path. The guarantee is scoped to a single composition and relies on the single-value identity per interface established in Section 3. Parts may hold mutable state under standard Java rules and dispatch adds no synchronization points.

6.5 Future Work

Per-interface composite roots as a general primitive. Assigning dispatch identity per interface at construction is, perhaps, a primitive that may be applied in areas beyond this paper's focus. It is an unexplored subject whether problems currently addressed through dynamic dispatch, AOP weaving, or flattened (or linearized) component hierarchies could benefit from the same mechanism.

Formal verification. A mechanized proof of Invariant 1 using small-step operational semantics would provide rigorous structural verification beyond the constructive proof sketch outlined in Section 3. Because the Parts model maps cleanly onto a core calculus, a modified subset of Featherweight Java serves as a natural starting point for formalizing these type-safety and execution guarantees.

Link timing and mutability. Through `final` links, composite identity is finalized at construction. This read-only guarantee is what makes sharing lock-free (Section 6): stable wiring needs no synchronization on the dispatch path. Two relaxations of the rule, opt-in *lazy* and *mutable* links, are worth exploring, and go in opposite directions.

A *lazy* link changes only the timing. Backed by the stable-value facility of recent JDK releases (Project Leyden), it wires on first use instead of at construction, yet stays written once and effectively final, so the guarantee holds while link initialization is deferred. A *mutable* link lifts the restriction. Mutating link assignments become legal, restoring the only aspect of conventional forwarding Parts disallows (runtime reconfiguration) but sacrifices the guarantee and under the same thread-safety obligations as a regular mutable field. Both relax restrictions of the current model rather than changing the underlying composition model. Whether a mutable link can also keep the guarantee, by confining rewiring to safe-publication points, is an open question.

Domain modeling and persistence integration. Normalized relational schemas naturally represent domain types as independently owned sets of data and relationships, whereas object-relational mapping often pressures those types into inheritance hierarchies. Table-per-hierarchy collapses types into a single hierarchy, often producing sparse, null-heavy tables; table-per-subclass preserves a more normalized representation but retains the coupling of inheritance. Conventional composition avoids that coupling but gives up open recursion. Parts is positioned to remove this tradeoff: independently modeled types can remain separate components, composed according to the domain's relationships while retaining open recursion across the composition. This suggests a domain model in which the object structure can mirror the normalized schema directly rather than introducing an inheritance hierarchy solely to retain open recursion. How such a mapping could be realized as a persistence framework is an open question.

7 Related Work

While Parts assumes static typing, this section also considers dynamically typed models where the structural comparisons remain conceptually instructive. The discussion is organized around the three properties that characterize Parts: independent runtime components, arbitrary composition, and open recursion.

Forwarding and conventional composition. Kotlin's by keyword [29], Lombok's `@Delegate` [30], Scala 3's `export` [28], and Go's struct embedding [31] provide forwarding from a composite to a supplied component. These mechanisms preserve independent runtime objects and permit the component structure to be composed freely. Self-calls, however, remain in the component's scope rather than dispatching through the composite. They therefore provide the first two properties but not open recursion.

Mixins and traits. Bracha and Cook [5] formalized mixins as parameterized abstract subclasses, while Schärli et al. [8] introduced traits as stateless, composable units of behavior. Odersky and Zenger [11] later introduced statically typed mixin composition with abstract type members. Scala's `with` clause linearizes traits into the target class at compile time, so self-calls dispatch through the outermost class in the resulting hierarchy. These systems therefore provide open recursion, but composition occurs by fusing the contributed behavior into a single class or object rather than by supplying an independent component at construction. A component implementation cannot consequently be selected or replaced independently at construction, nor can a single component instance participate independently in multiple composites. This fusion is also characteristic of recent precisely-typed mixin systems [12]. Parts separates the two: open recursion is provided by interface-scoped dispatch without fusing the component into the composite.

Chugh's ISOLATE [13] gives a statically typed calculus for functional objects in which *premethods* are defined apart from a host object and later mixed into it. ISOLATE provides typed open recursion without an explicit fixpoint, but the resulting object is still assembled by mixin composition rather than from an independently supplied runtime component. It therefore occupies the same side of the tradeoff: open recursion without independent runtime components.

Family polymorphism and dynamic composition. Family-oriented systems such as gbeta [20, 22], CaesarJ [23], Object Teams/Java [24], and J& [21] support extensible families whose members can participate in open recursive composition. Their object structure, however, is defined by the family model: members are instantiated as part of the family rather than supplied as independent runtime components.

Newspeak [25] makes names late-bound and permits classes to be supplied as first-class constructor arguments, supporting runtime wiring and reconfiguration of module objects. Its open recursion derives from mixin inheritance [5]: classes are applied to superclasses and fused into a single object. Composed module objects are instead held by reference and reached through ordinary message sends, so self-calls do not route through the composition. Newspeak therefore offers both dynamic composition and open-recursive inheritance, but not open recursion across independently composed runtime objects.

Delegation. Several research systems have explored delegation as a way to retain independent runtime objects while restoring open recursion.

Kniesel's DARWIN [15, 16] provides statically typed delegation-based single implementation inheritance. A base object is supplied at run time through a `delegatee` field, and self-calls route through the inheriting subtype. DARWIN permits at most one delegating or forwarding field per class [16, §6.5]; multiple delegation is deliberately excluded [16, §6.5.1]. A receiver typed through a

single interface therefore cannot represent multiple independently rooted compositions [16, §5.5.4]. DARWIN provides independent components and open recursion, but is effectively limited to the single-root, single-source topology (a).

Incomplete Featherweight Java (IFJ) [18] similarly provides static typing, independent runtime objects, and open recursion through *incomplete objects* completed by other objects at run time. Its delegation is structural and per method, however, with a single `this` binding for the delegation chain [18, §1, Fig. 1]. As we read it, a component therefore cannot root different interfaces at different composites. Its composition is consequently confined to topology (a) (§3.3). Parts instead associates each interface with a specific composite through `@link`, permitting the multi-rooted and multi-source topologies induced by partial delegation.

DelphJ [17] extends the Java metaprogramming DSL MorphJ with delegation through *access-paths*, supplying runtime *subobjects* with late-bound self-calls. As with DARWIN and IFJ, its dispatch is caller-side: a self-call recovers its composite context by traversing an access path. The path is carried with the call, bound to `this` at invocation, built and merged during field reads, and looked up on dispatch [17, §3], so every invocation pays to maintain the path and to traverse it, and the cost grows with its length. The mechanism has no per-interface identity, so as we read it, a self-call to a method shared by two separately claimed interfaces cannot be routed between the composites claiming them (topologies (b),(d); Figure 7), and the multi-source case (topology (c), Figure 5) cannot dispatch self-calls correctly between separately claimed interfaces. DelphJ's composition is consequently confined to topology (a) (§3.3). ISD instead records identity once at construction, off the dispatch path, and resolves it from a single slot.

Delegation models that emulate inheritance's tree-shaped hierarchies through caller-side dispatch are generally bound to topology (a). Multi-rooted topologies ultimately require the identity of the composite that claims the interface, which a caller-side strategy cannot produce from the information available where a call enters the structure. Parts inverts the caller-side assumption: composite identity is fixed in the component at construction rather than reconstructed by the caller on every access. This design choice is what allows delegation, with its late-bound self, to support the topologies that forwarding already permits.

Delegation at larger granularity. Ostermann's delegation layers [19] extend delegation from individual objects to entire collaborations composed at run time. Operations are routed to the composed collaboration rather than to a single layer, providing "composition consistency." The unit of composition is a collaboration, however, whose participants are nested classes redirected through the enclosing instance. The model therefore provides open recursion at collaboration granularity rather than through independently supplied components, and does not provide the per-component partial delegation that gives rise to multi-rooted and multi-source topologies.

Prototype-based delegation provides another form of open recursion. Lieberman [3] introduced *open delegation*, in which an object delegates unanswered messages to a prototype while `self` remains the original receiver; Self [9] realized this model in a working language. These systems support late-bound components and open recursion, but delegation is object-granular rather than interface-scoped. A prototype supplies all behavior not handled by the delegating object, so partial delegation and the resulting multi-rooted topologies are not expressible. Their dynamic typing also provides no static interface contract for the composed result. Cecil [10] explores a statically typed point in this space, separating subtyping from code inheritance, but does not provide dynamic inheritance of the kind relevant here.

Subject-oriented programming [26, 27] is particularly relevant to multi-source composition. It composes independent *subjects*, each representing a complete view of a shared domain, using externally specified composition rules such as `merge` and `override`. The model explicitly targets

decentralized composition in which no subject is the sole owner of the result. This corresponds closely to the multi-source structure of topology (c). The original model and its implementations, however, do not preserve independently composed runtime objects in the final representation and do not provide static typing across the composed result [27]. Parts addresses the same structural problem at the granularity of an individual runtime object, using interface-scoped identity under a statically checked interface contract.

Multiple inheritance. C++ virtual base classes and virtual inheritance [14] resolve inheritance diamonds within a single object and its merged class hierarchy. They therefore address ambiguity among inherited implementations rather than composition among independently supplied runtime components. Multiple inheritance does not provide the independent component identity or dynamic composition considered by Parts.

Novelty. The surveyed systems divide naturally according to the three properties of Parts. Forwarding provides independent runtime components and arbitrary composition, but not open recursion. Mixins, traits, family systems, and related forms of class composition provide open recursion, but fuse the components into a statically determined object structure. Delegation systems such as DARWIN, IFJ, and DelphJ restore open recursion while retaining independent runtime components, but restrict the composition topologies they can represent.

The closest prior systems are therefore the delegation models. DARWIN deliberately excludes multiple delegation [16, §6.5.1]; IFJ binds a single `this` across its delegation structure; and DelphJ's access-path mechanism, the most capable of the three, still limits composition to only the simplest single-rooted case. Of the three properties, arbitrary composition is the practically unobtainable one under these caller-side delegation models.

Parts unifies the three properties through interface-scoped dispatch. By assigning composite identity per interface at construction, ISD allows a single component to participate in the topologies that forwarding already permits while preserving late-bound self-calls. The resulting self-dispatch path has $O(1)$ cost and, as implemented, is empirically equivalent to a standard vtable call (Section 5).

8 Conclusion

The familiar advice to favor object composition over class inheritance leaves an important tradeoff unresolved: composition gives up the open recursion that makes inheritance so effective for extension. Parts resolves the tradeoff. By assigning composite identity per interface at construction, it shows that open recursion need not depend on inheritance's single runtime object, and that the two can be decoupled in practice. The result is a general model that retains the essential properties of simple forwarding, namely independent runtime components and arbitrary composition topologies, while adding open recursion at vtable speed.

References

[1] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

[2] L. Mikhajlov and E. Sekerinski. A study of the fragile base class problem. In *Proceedings of ECOOP*, LNCS 1445, pages 355–382, 1998.

[3] H. Lieberman. Using prototypical objects to implement shared behavior in object-oriented systems. In *Proceedings of OOPSLA*, pages 214–223, 1986.

[4] K. C. Sekharaiah and D. Janaki Ram. Object schizophrenia problem in modeling is-role-of inheritance. In *Proceedings of the ECOOP Workshop on Inheritance*, Málaga, Spain, 2002.

[5] G. Bracha and W. Cook. Mixin-based inheritance. In *Proceedings of OOPSLA/ECOOP*, pages 303–311, 1990.

[6] W. Cook and J. Palsberg. A denotational semantics of inheritance and its correctness. In *Proceedings of OOPSLA*, pages 433–443, 1989.

[7] W. R. Cook, W. Hill, and P. S. Canning. Inheritance is not subtyping. In *Conference Record of POPL*, pages 125–135, 1990.
[8] N. Schärli, S. Ducasse, O. Nierstrasz, and A. Black. Traits: Composable units of behaviour. In *Proceedings of ECOOP*, LNCS 2743, pages 248–274, 2003.
[9] D. Ungar and R. B. Smith. Self: The power of simplicity. In *Proceedings of OOPSLA*, pages 227–242, 1987.
[10] C. Chambers. The Cecil language: Specification and rationale. Technical Report UW-CSE-93-03-05, University of Washington, 1993.
[11] M. Odersky and M. Zenger. Scalable component abstractions. In *Proceedings of OOPSLA*, pages 41–57, 2005.
[12] A. Fan and L. Parreaux. super-charging object-oriented programming through precise typing of open recursion. In *Proceedings of ECOOP*, LIPIcs 263, pages 11:1–11:28, 2023. DOI: 10.4230/LIPIcs.ECOOP.2023.11.
[13] R. Chugh. ISOLATE: A type system for self-recursion. In *Proceedings of ESOP*, LNCS 9032, pages 257–282, 2015. DOI: 10.1007/978-3-662-46669-8_11.
[14] B. Stroustrup. Multiple inheritance for C++. In *Proceedings of the EUUG Spring Conference*, 1987.
[15] G. Kniesel. Type-safe delegation for run-time component adaptation. In *Proceedings of ECOOP*, LNCS 1628, pages 351–366, 1999. DOI: 10.1007/3-540-48743-3_16.
[16] G. Kniesel. *Dynamic Object-Based Inheritance with Subtyping*. PhD thesis, University of Bonn, 2000.
[17] P. Gerakios, A. Biboudis, and Y. Smaragdakis. Forsaking inheritance: Supercharged delegation in DelphJ. In *Proceedings of OOPSLA*, pages 233–252, 2013.
[18] L. Bettini, V. Bono, and B. Venneri. Delegation by object composition. *Science of Computer Programming*, 76:992–1014, 2011. DOI: 10.1016/j.scico.2010.04.006.
[19] K. Ostermann. Dynamically composable collaborations with delegation layers. In *Proceedings of ECOOP*, LNCS 2374, pages 89–110, 2002.
[20] E. Ernst, K. Ostermann, and W. R. Cook. A virtual class calculus. In *Conference Record of POPL*, pages 270–282, 2006.
[21] N. Nystrom, X. Qi, and A. C. Myers. J&: Nested intersection for scalable software composition. In *Proceedings of OOPSLA*, pages 347–366, 2006.
[22] E. Ernst. Family polymorphism. In *Proceedings of ECOOP*, LNCS 2072, pages 303–326, 2001.
[23] I. Aracic, V. Gasiunas, M. Mezini, and K. Ostermann. An overview of CaesarJ. In *Transactions on Aspect-Oriented Software Development I*, LNCS 3880, pages 135–173, 2006.
[24] S. Herrmann. Object Teams: Improving modularity for crosscutting collaborations. In *Proceedings of NetObjectDays*, LNCS 2591, pages 248–274, 2002.
[25] G. Bracha, P. von der Ahé, V. Bykov, Y. Kashai, W. Maddox, and E. Miranda. Modules as objects in Newspeak. In *Proceedings of ECOOP*, LNCS 6183, pages 405–428, 2010.
[26] W. Harrison and H. Ossher. Subject-oriented programming (a critique of pure objects). In *Proceedings of OOPSLA*, pages 411–428, 1993.
[27] H. Ossher, M. Kaplan, W. Harrison, A. Katz, and V. Kruskal. Subject-oriented composition rules. In *Proceedings of OOPSLA*, pages 235–250, 1995.
[28] EPFL. Exports — Scala 3 reference. https://docs.scala-lang.org/scala3/reference/other-new-features/export.html, 2024.
[29] JetBrains. Delegation — Kotlin programming language. https://kotlinlang.org/docs/delegation.html, 2024.
[30] Project Lombok. `@Delegate` — Lombok features. https://projectlombok.org/features/Delegate, 2024.
[31] The Go Authors. Embedding — Effective Go. https://go.dev/doc/effective_go#embedding, 2024.
[32] A. Shipilev and contributors. JMH: Java Microbenchmark Harness. https://github.com/openjdk/jmh, 2013–2024.
[33] R. S. McKinney. manifold-parts. https://github.com/manifold-systems/manifold/tree/master/manifold-deps-parent/manifold-parts, 2026.